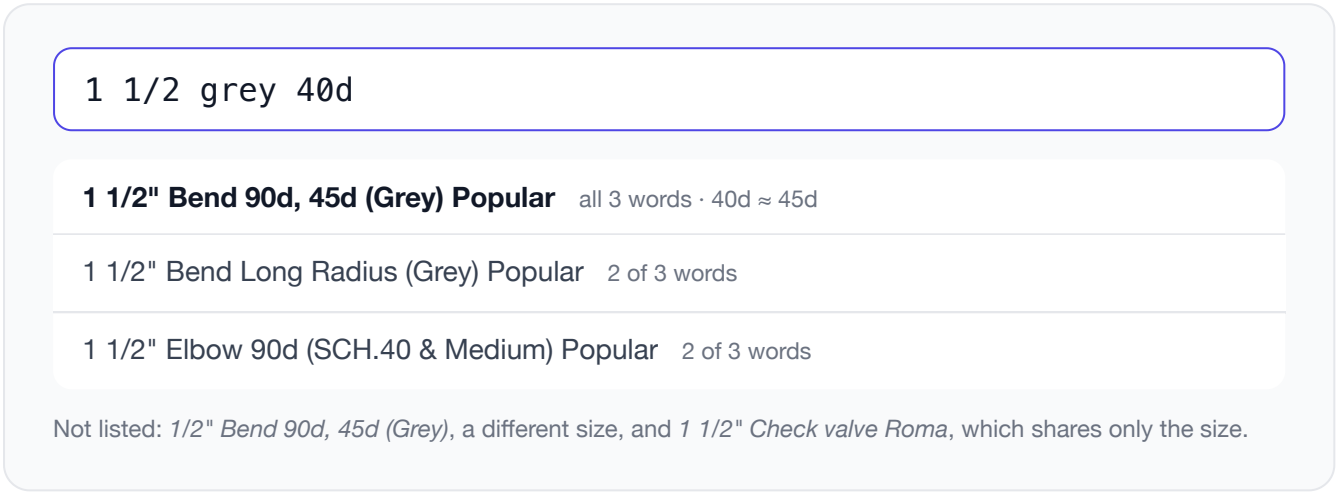
POS · PRODUCT SEARCH

How smart search works

A plain-language guide to the search on your cashier counter and products screen: how it finds *1 1/2" Bend 90d, 45d (Grey) Popular* when you type `1 1/2 grey 40d`, and why it is fast.



1 The problem with `.includes()`
2 The big picture
3 Step 1 · Clean and split into words
4 Step 2 · Match each word
5 Typos and edit distance
6 When is a typo allowed?
7 The rules: sizes, and how many words
8 Points and bonuses
9 Rare words count more
10 Sorting the results
11 The whole example, worked out
12 The second chance
13 Making it fast
14 Where it lives in your code
15 Where to go next · Glossary

Every number in this guide comes from running the real search code.

1 The problem with `.includes()`

The usual first search compares one long string with another. Shop names break it.

```
const found = products.filter(p =>
  p.name.toLowerCase().includes(query.toLowerCase()));
```

`.includes()` asks one question: **do these exact characters appear, in this order, side by side?**

NAME	YOU TYPE	.INCLUDES()	WHY
1 1/2" Bend 90d, 45d (Grey) Popular	`1 1/2 grey 40d`	**no**	Words in a different order, a quote mark in the way, and 40d is not 45d.
1 1/2" Bend 90d, 45d (Grey) Popular	`grey bend`	**no**	"Bend" comes before "Grey" in the name.
1 1/2" Bend 90d, 45d (Grey) Popular	`1/2 bend`	**wrong yes**	`1/2` is inside `1 1/2`, so a different size shows up.

So `.includes()` misses what you want and shows what you don't. Making it a bit cleverer, for example checking whether a name *starts with* the query, fixes a few cases but not word order or typos.

> **The big idea.** Stop comparing one long string with another. Break both into **words**, match the words one by one, give each product a **score** for how well it matched, and list the best scores first.

This is how real search engines work, from the one on Amazon to Google's. Yours is a small, focused version tuned for a hardware shop's catalogue: sizes, fittings and brands.

2 The big picture

Five steps, run on every product name, every time you type.

1 — Clean & split
Lower case, drop punctuation, cut into words

›

2 — Match words
Each typed word looks for its best partner in the name

›

3 — Check rules
Sizes must match; you may miss one word in three

›

4 — Score
Points for each match, more for rare words

›

5 — Sort
Most words matched first, then most points

The next pages take these one at a time, using the products from the cover.

3 Step 1 · Clean and split into words

Computers compare characters, so first make names and searches look alike. This is called **normalising**, and the splitting is called **tokenising**.

1. **Lower case everything.** "Grey", "GREY" and "grey" become the same.
2. **Turn punctuation into spaces.** Quotes, brackets, commas and `&` go. Keep `/`, `-` and `.`, because they are part of sizes like `1/2` and `sch.40`.
3. **Glue a whole number to a fraction.** `1 1/2` becomes `1-1/2`, so it is one size, not the two words "1" and "1/2". Written `1-1/2`, it ends up the same.
4. **Split on spaces.**

Product name	`1 1/2" Bend 90d, 45d (Grey) Popular`
Its words	`1-1/2` `bend` `90d` `45d` `grey` `popular`
You type	`1 1/2 grey 40d`
Typed words	`1-1/2` `grey` `40d`
Another name	`1/2" Bend 90d, 45d (Grey) Popular`
Its words	`1/2` `bend` `90d` `45d` `grey` `popular`

Why gluing sizes matters. After this step, `1/2` and `1-1/2` are simply different words, so searching `1/2` can never pick up a 1 1/2" pipe. Step 1 alone fixes the "wrong size" problem from section 1.

```
// server/services/smartSearch.ts (simplified)
function searchWords(text) {
  return text.toLowerCase()
    .replace(/[^\p{L}\p{N}/.-]+/gu, ' ')            // punctuation → space
    .replace(/(^|\s)(\d+)[\s-]+(\d+\/\d*)/g, '$1$2-$3') // 1 1/2 → 1-1/2
    .split(/\s+/)
    .filter(Boolean);
}
```

4 Step 2 · Match each word

Each typed word looks through the name's words for its best partner. There are four kinds of match, each worth some points.

KIND	EXAMPLE	POINTS	WHEN
Same word	`grey` = `grey`	3	Always
Starts the word	`elb` → `elbow`	2	Always; this is what makes search-as-you-type work
One typo away	`40d` → `45d`	1	Only for words that may be typos (section 6)
Inside the word	`x24` in `25x24`	1	Only as a last resort (section 12)

Each name word can be used once. If you type `1 1`, the two 1s can't both claim the same word. Matching is greedy: each typed word, in order, takes the best word still free.

Matching `1 1/2 grey 40d` against two names

1 1/2" Bend 90d, 45d (Grey) Popular

`1–1/2 · same · 3` `bend` `90d · typo · 1` `45d` `grey · same · 3` `popular`

1/2" Bend 90d, 45d (Grey) Popular

`1–1/2 · no match` `1/2` `bend` `90d · typo · 1` `45d` `grey · same · 3`

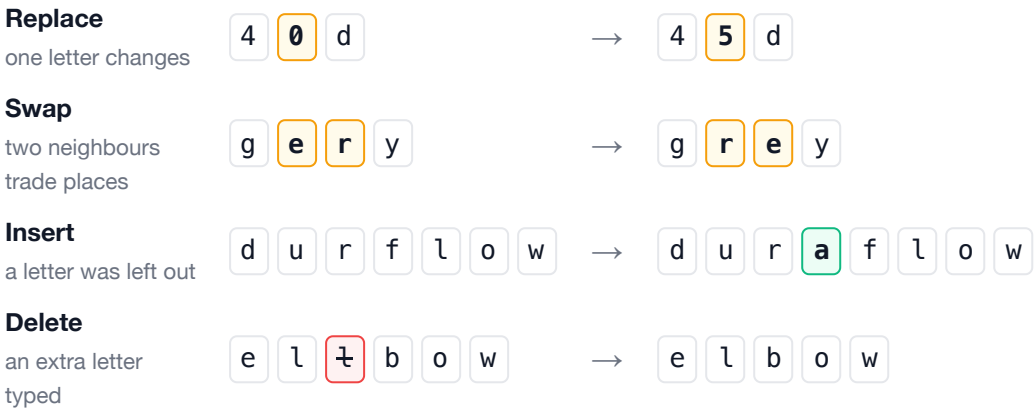
"40d" is one keystroke from both 90d and 45d. The first one found wins, which makes no difference to the score.

The second product matched two of the three words, but one it missed is a **size**. The rules in section 7 throw it out.

5 Typos and edit distance

How does a computer know "40d" is close to "45d" but "40d" is far from "elbow"? It counts the smallest number of single-letter changes, called **edits**, that turn one word into the other.

This count is the **edit distance**, also known as Levenshtein distance; the version that counts swaps is called Damerau–Levenshtein. Four kinds of edit count as one:



Your search allows **one edit**, or two for words of 8 letters or more. One edit catches nearly every real slip of the finger.

The fast way to check "one edit or less"

The textbook method fills a table with one row per letter of one word and one column per letter of the other. It is correct, but doing that for every word of every product was slow: up to 600 ms per search on 3,000 products. For "one edit", there's a shortcut:

1. Walk both words from the left while the letters agree.
2. At the first difference, try each fix once: *replace* (skip a letter in both), *swap* (the next two are crossed), *insert/delete* (skip a letter in one word).
3. If the rest of the two words is then identical, it's one edit.

```
function withinOneEdit(a, b) {
  if (Math.abs(a.length - b.length) > 1) return false;
  let i = 0;
  while (i < a.length && i < b.length && a[i] === b[i]) i++;   // same so far
  if (a.length === b.length)                                    // replace or swap
    return a.slice(i + 1) === b.slice(i + 1)
        || (a[i] === b[i + 1] && a[i + 1] === b[i] && a.slice(i + 2) === b.slice(i + 2));
  return a.length > b.length                                    // insert or delete
    ? a.slice(i + 1) === b.slice(i)
    : a.slice(i) === b.slice(i + 1);
}
```

One pass along the word, with no table. Together with remembering each name's words (section 13), this took the slowest search from about 600 ms to 35 ms.

6 When is a typo allowed?

Typo tolerance is powerful but dangerous: allowed everywhere, it matches nonsense. Guard rails decide where it's used.

RULE	EXAMPLE THE RULE STOPS	WHY
The typed word has 4+ characters, or 3 with a digit	`tee` ≈ `tea`, `the`	Short words are one edit from too many others.
The name word must also pass that rule	`bend` → `end` (End Cap)	The edit check says yes (delete "b"), but a 3-letter word is too short to trust.
Never for sizes	`25mm` → `20mm`, `1/2` → `1/4`	One digit apart is a *different product*, not a typo.
At least one letter in the word	`2024` → `2025`	Plain numbers are sizes, quantities or codes, so they must be exact.

> **Lesson:** the clever part of a smart search is often deciding where *not* to be clever. `40d` → `45d` helps the cashier; `25mm` → `20mm` sells the wrong fitting.

Typos in a half-typed word

While typing "duraflow" you might have `durfl` so far, with the "a" missed. The whole word is too far off, so the search also compares against the *start* of each name word (`durafl`), allowing one edit there too.

7 The rules: sizes, and how many words

After matching, two rules decide whether a product is listed at all.

Rule 1 · A typed size must match

A size is `1/2`, `1-1/2`, `25`, `25mm` and so on. If you typed one and the product doesn't have it, it's out, however well the other words match.

That's why *1/2" Bend 90d, 45d (Grey)* is not listed for `1 1/2 grey 40d`, even though "grey" and "40d" match.

Rule 2 · Miss at most one word in three

To show *similar* products, a product may miss some typed words. It must still match `n - ⌊n/3⌋` of the `n` words you typed:

WORDS TYPED	1	2	3	4	5	6
Must match	1	2	2	3	4	4

So with one or two words, everything must match, which keeps short searches precise. With three or more, a near miss can appear, but always **below** the products that match everything (section 10).

> **Precision vs. recall.** Search people talk about two goals. **Precision**: everything shown is relevant. **Recall**: everything relevant is shown. Strict rules (sizes, short searches) protect precision; the one-in-three allowance and typos add recall. Ranking lets you have both, because the extras sit at the bottom.

8 Points and bonuses

Every product that passes the rules gets a score. Higher means a better match.

SOURCE	POINTS	IDEA
Each typed word's match (same / starts / typo)	3 / 2 / 1 × weight	Better matches score more; the weight is section 9.
The product's *first* word was matched	+2	Your names start with the size ("25mm Clamp Duraflow"), so matching it is a strong sign.
Each word found in the order you typed	+0.5	"bend grey" fits "Bend … Grey" a little better than "Grey … Bend".
Barcode or SKU *starts with* the query	+2, full match	Typing the start of a code finds the product.
Barcode or SKU *is* the query	top of the list	A scanned or typed code is a certain answer.

9 Rare words count more

In a hardware shop, "Popular" is a brand on hundreds of products, so it tells you very little. "Tee" is on a few, so it tells you a lot.

Each typed word gets a **weight** based on how many of the found products contain it:

```
weight = 1 + ln( (products found + 1) ÷ (products with this word + 1) )
```

"ln" is the natural logarithm: it grows slowly, so a rare word counts more without drowning out everything else. A word every product has gets weight 1.

Searching `popular tee 25mm` among some 25mm fittings, 6 products pass the rules:

TYPED WORD	PRODUCTS WITH IT	WEIGHT
`popular`	5 of 6	1.15
`tee`	2 of 6	**1.85**
`25mm`	6 of 6	1.00

PRODUCT	MATCHED	SCORE
25mm Tee Popular	popular, tee, 25mm	14.50
25mm Tee Duraflow	tee, 25mm	11.05
25mm Socket Popular	popular, 25mm	8.95

Without weights, the Socket and the Tee Duraflow would tie. With them, the other *tee* comes next, which is what a cashier means. This idea is called **IDF** (inverse document frequency), and it's at the heart of the big search engines' ranking formulas (TF-IDF, BM25).

10 Sorting the results

Products are compared on these, in order; each one only matters when everything above it is equal.

1. **Barcode or SKU typed exactly:** first.
2. **More typed words matched:** a product matching 3 of 3 always beats 2 of 3, whatever the points. This rule keeps "similar" products below exact ones.
3. **More points.**
4. **Shorter name:** fewer extra words means a more direct match.
5. **Natural A–Z:** numbers by value, so `3"` comes before `24"`. A plain sort puts "24" first because "2" < "3". JavaScript's `Intl.Collator('en', {numeric: true})` does this for you.

11 The whole example, worked out

You type `1 1/2 grey 40d`. Typed words: `1-1/2` `grey` `40d`. Five products, three found.

Weights (3 products found): `1-1/2` is in all 3 → **1.00**; `grey` matched in 2 → **1.29**; `40d` matched (as a typo) in 2 → **1.29**.

PRODUCT	1-1/2	grey	40d	BONUS	SCORE	RESULT
1 1/2" Bend 90d, 45d (Grey) Popular	3 × 1.00	3 × 1.29	1 × 1.29	3	**11.15**	**1st** · all 3 words
1 1/2" Bend Long Radius (Grey) Popular	3 × 1.00	3 × 1.29	—	3	9.86	2nd · 2 of 3
1 1/2" Elbow 90d (SCH.40 & Medium) Popular	3 × 1.00	—	1 × 1.29	3	7.29	3rd · 2 of 3
1/2" Bend 90d, 45d (Grey) Popular	missed	a typed size is missing				not listed
1 1/2" Check valve Roma	3	—	—	1 of 3; needs 2		not listed

Bonus 3 = +2 for matching the first word (the size) + 0.5 for each of two words found in the order typed.

12 The second chance

Sometimes nothing matches by whole words or word starts, for example a search for the middle of a code.

If the first pass finds **nothing at all**, the search runs again and also accepts:

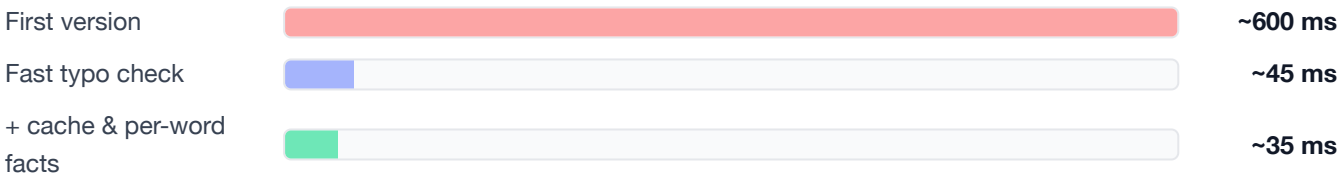
- a typed word **inside** a name word (`x24` inside `25x24`);
- the whole query anywhere in the name, the old `.includes()`, now only as a fallback;
- the query inside a barcode or SKU.

Running it only when needed matters: if "inside" matches were always on, `25` would match `125mm`, and short searches would fill with noise.

13 Making it fast

The search scores every product on every keystroke, so speed matters. Four things keep it quick.

1. **Load only what's needed to rank.** The server asks the database for just name, barcode and SKU, not prices, images or stock. That's a few thousand short rows. Only the products actually shown (one page) are then loaded in full.
2. **Remember each name's words.** Splitting 3,000 names on every keystroke is wasted work, since names rarely change. The words are kept in memory (a *cache*) after the first search.
3. **The fast typo check** from section 5.
4. **Work out per-word facts once.** Whether a typed word may be a typo is decided once per search, not once per product.



Slowest search, 3,000 products on a Mac. Most searches take 15–20 ms; the price-list catalogue of about 450 products takes 2–10 ms.

> **Measure, don't guess.** The first version was correct but too slow, and that only showed up by timing it on 3,000 products. Every improvement above was chosen by measuring where the time went.

14 Where it lives in your code

FILE	JOB
`server/services/smartSearch.ts`	The search itself: words, matching, typos, rules, scores, sorting. Pure code with no database, easy to test.
`electron/src/search/smartSearch.ts`	The same file for the desktop app, so the till searches identically offline. A test fails if the two copies ever differ.
`server/services/productSearchService.ts`	Loads names from MongoDB, ranks them, then loads the products to show (all for the counter, one page for the products screen).
`electron/src/ipc/products.ts`	The same for the desktop app's SQLite database.
`server/tests/smartSearch.test.ts`	Tests: any word order, typos, sizes stay exact, barcode first, both copies identical.

15 Where to go next

Your search is a small, hand-built version of what dedicated search engines do. Some ideas to explore:

- **Synonyms:** a list saying `bend` ≈ `elbow` or `grey` = `gray`, checked in step 2.
- **Highlighting:** step 2 already knows which words matched, so the counter could show them in bold.
- **Inverted index:** like the index at the back of a book, a map from each word to the products containing it. Instead of scoring every product, look up only those containing the typed words. Essential at hundreds of thousands of products.
- **BM25:** the refined version of the rare-word weighting used by most search engines.
- **Search engines:** Meilisearch, Typesense, Elasticsearch, and MongoDB Atlas Search have all of this built in. Worth it once you have tens of thousands of products or many shops searching at once.

Glossary

Normalising

Making text uniform before comparing: lower case, punctuation removed, sizes written one way.

Tokenising / tokens

Cutting text into words ("tokens").

Edit distance (Levenshtein, Damerau–Levenshtein)

The fewest single-letter changes (replace, insert, delete, plus swap for Damerau) turning one word into another.

Precision / recall

Precision: what's shown is relevant. Recall: what's relevant is shown.

IDF (inverse document frequency)

Weighting words by rarity: rare words say more about what you want.

Ranking

Ordering results by score instead of just filtering them.

Fallback

A looser second pass, used only when the strict one finds nothing.

Natural sort

Sorting numbers inside text by value (3" before 24").

Cache

Keeping a computed result in memory to reuse instead of recomputing.